

Reviewer Pack — Minimal Rank of Tensor Product of Tropicalizations Bounds XO...

Entry 5eea1bb749c7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 11:37:05 UTC

Minimal Rank of Tensor Product of Tropicalizations Bounds XOR-AND Tree Width

Entry ID: 5eea1bb749c7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 11:37:05 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Tensor Products) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For every CNF formula F , let $T(F)$ be its associated tropical curve in the max-plus semiring. Let T_F denote the tensor product of $T(F)$ with itself. Then, the XOR-AND tree width $t(T_F)$ is at least a polynomial function of the rank of $T(F)$. Equivalently: for all CNFs F , there exists a constant $c > 0$ such that $t(T_F) \geq c * |T_F|$.’]

Rationale (proposer’s reasoning):

[‘The tensor product operation in tropical geometry is known to increase the complexity of tropical varieties. If this increase is polynomially related to the XOR-AND tree width, it suggests a deep connection between the algebraic structure of CNF formulas and their computational hardness.’]

Taxonomy category: TROPICALGEOMETRY_XORANDTREEWIDTH (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a23d2b9d4d4e7469

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient between the tensor product’s rank and XOR-AND tree width for all CNFs exceeds 0.8, and the mean XOR-AND tree width is at least 3 standard deviations above the median of all computed widths.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tensor product tropical geometry AND XOR-AND tree width - max-plus semiring tensor product rank CNF formula - polynomial bound on XOR-AND tree width tropical curve

Top relevant hits considered: - [<http://arxiv.org/abs/math/0208014v2>] Rational semimodules over the max-plus semiring and geometric approach of discrete event systems - [<http://arxiv.org/abs/1503.00822v2>] Product Ranks of the 3×3 Determinant and Permanent - [<http://arxiv.org/abs/2407.09775v1>] Learning Weighted Finite Automata over the Max-Plus Semiring and its Termination

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction
from itertools import product

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
    for i in range(m):
        max_row = i + max(range(i, m), key=lambda r: abs(A[r][i]))
        A[i], A[max_row] = A[max_row], A[i]
        pivot = A[i][i]
        if pivot == 0:
            continue
        for j in range(n):
            A[i][j] /= pivot
        for k in range(m):
            if k != i and A[k][i] != 0:
                factor = A[k][i]
                for j in range(n):
                    A[k][j] -= factor * A[i][j]
    return A

def matrix_rank(A):
    rank = 0
    A_copy = [row[:] for row in A]
    m, n = len(A), len(A[0])
    for i in range(min(m, n)):
        if A_copy[i][i] != 0:
            rank += 1
```

```

        for j in range(i + 1, m):
            factor = A_copy[j][i] / A_copy[i][i]
            for k in range(n):
                A_copy[j][k] -= factor * A_copy[i][k]
    return rank

def xor_and_tree_width(clauses):
    n = len(clauses[0])
    assignments = list(product([False, True], repeat=n))
    satisfied = 0
    unsatisfied = 0
    for assignment in assignments:
        if any(all(assignment[j-1] if c > 0 else not assignment[j-1] for c in clause) for clause in clauses):
            satisfied += 1
        else:
            unsatisfied += 1
    return max(satisfied, unsatisfied)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    m = random.randint(n, n * (n - 1))

    clauses = []
    for _ in range(m):
        variables = set(random.sample(range(1, n + 1), random.randint(1, n)))
        clause = [random.choice([1, -1]) * v for v in variables]
        clauses.append(clause)

    T_F_rank = matrix_rank(gaussian_elimination([[sum(c[i] for c in clause) for i in range(n)] for clause in clauses]))
    width = xor_and_tree_width(clauses)

    return {
        "metric_name": "XOR-AND tree width",
        "metric_value": width,
        "instances_tested": len(clauses),
        "conjecture_holds": T_F_rank * 3 > width,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 89))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_width = sum(r["metric_value"] for r in results) / len(results)
    std_width = math.sqrt(sum((r["metric_value"] - mean_width) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_width} std={std_width} support_fraction={support_fraction}")
else:
    first_failing_seed = next((seed for seed, r in enumerate(results) if not r["conjecture_holds"]), None)
    print(f"RESULT: FALSIFIED counterexample=\"first failing seed\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e47b9692.py", line 87, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e47b9692.py", line 70, in run_trial
    T_F_rank = matrix_rank(gaussian_elimination([[sum(c[i] for c in clause) for i in range(n)] for clause in clauses]))
                  ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e47b9692.py", line 70, in <genexpr>
    T_F_rank = matrix_rank(gaussian_elimination([[sum(c[i] for c in clause) for i in range(n)] for clause in clauses]))
                  ~~~~~^
TypeError: 'int' object is not subscriptable

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from evaluating the conjecture's support conditions. | next: Investigate and fix the error in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14354
2	preregistration	ollama_remote	glm4:latest	0	0	6011
3	novelty	ollama_remote	glm4:latest	0	0	4548
4	novelty	ollama_remote	glm4:latest	0	0	6059
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26947
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12327
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15555
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10586

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	8195

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 104581 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/5eea1bb749c7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5eea1bb749c7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5eea1bb749c7.tar.gz (if generated)